Univerzitet u Sarajevu
Elektrotehnički fakultet
Predmet: Uvod u digitalnu obradu slike
Predmetni profesor: prof. dr Emir Sokić
Autor: Berina Biberović
Akademska godina: 2022/2023.

Tutorijal 6

Template matching

Template matching ili podudaranje predložaka je metoda za tražnje i nalaženje slike predloška u većoj slici. OpenCV ima funkciju `cv::matchTemplate()` koja radi upravo to. Ta funkcija stavlja sliku predloška preko ulazne velike slike (kao 2D konvolucija) i upoređuje predložak i djelić ulazne slike ispod predloška. Kako predložak treba biti pravougaonik, može biti da nije cijeli pravougaonik relevantan za traženje onoga što nam treba iz velike ulazne slike - u tom slučaju može se koristiti maska da se izoluje dio predloška koji se treba koristiti da se nađe podudaranje. Detaljnije ćemo kasnije pričati o ovome.

Sljedeći primjer ilustruje kako uz pomoć šablona (tj. predloška) možemo naći da li se na ulaznoj slici nalazi traženi šablon.

```cpp
#include <opencv4/opencv2/imgcodecs.hpp>
#include <opencv4/opencv2/highgui.hpp>
#include <opencv4/opencv2/imgproc.hpp>
#include <opencv4/opencv2/core.hpp>

int main() {

    // Ucitavanje ulazne slike:
    cv::Mat slika = cv::imread("../Moje karte/IMG_0002.JPG",
        cv::IMREAD_COLOR);
    // Ucitavanje predloska:
    cv::Mat temp = cv::imread("../Moje karte predlosci/herc.JPG",
        cv::IMREAD_COLOR);

    cv::Mat rezultat;

    // Ove varijable su nam potrebne za funkciju za template matching:
    int rezcol, rezrow;
    rezcol = slika.cols - temp.cols + 1;
    rezrow = slika.rows - temp.rows + 1;

    rezultat.create(rezrow, rezcol, CV_32FC1);

    cv::matchTemplate(slika, temp, rezultat, cv::TM_CCORR_NORMED);

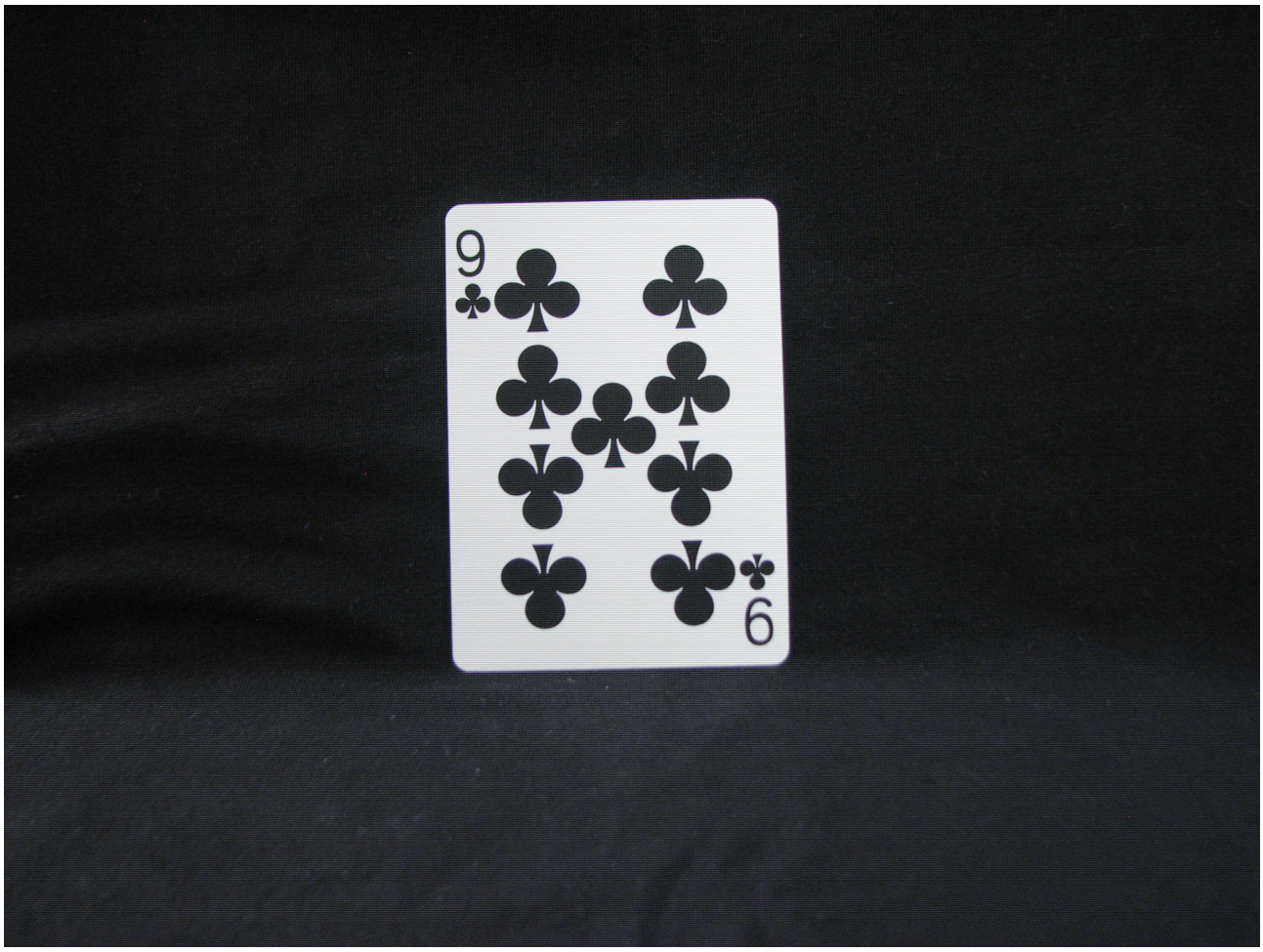
    // Sad racunamo koliko se podudaraju i gdje:
    double min_vr, max_vr;
    cv::Point min_lok, max_lok, match_lok;

    cv::minMaxLoc(rezultat, &min_vr, &max_vr, &min_lok, &max_lok);
    match_lok = max_lok;

    // Sad trebamo nacrtati gdje je template nadjen:
    cv::rectangle(slika, match_lok, cv::Point(match_lok.x + temp.cols,
        match_lok.y + temp.rows), cv::Scalar(0, 0, 255), 2, 8, 0);
```

```
33
34      cv::imshow("Rezultat template matchinga", slika);
35      cv::waitKey();
36
37      return 0;
38  }
```

U prethodnom primjeru su korištene igraće karte kako bi se demonstriralo kako radi *template matching* u OpenCV-u. Ulazna slika je prikazana na slici 1, a šablon je prikazan na slici 2.

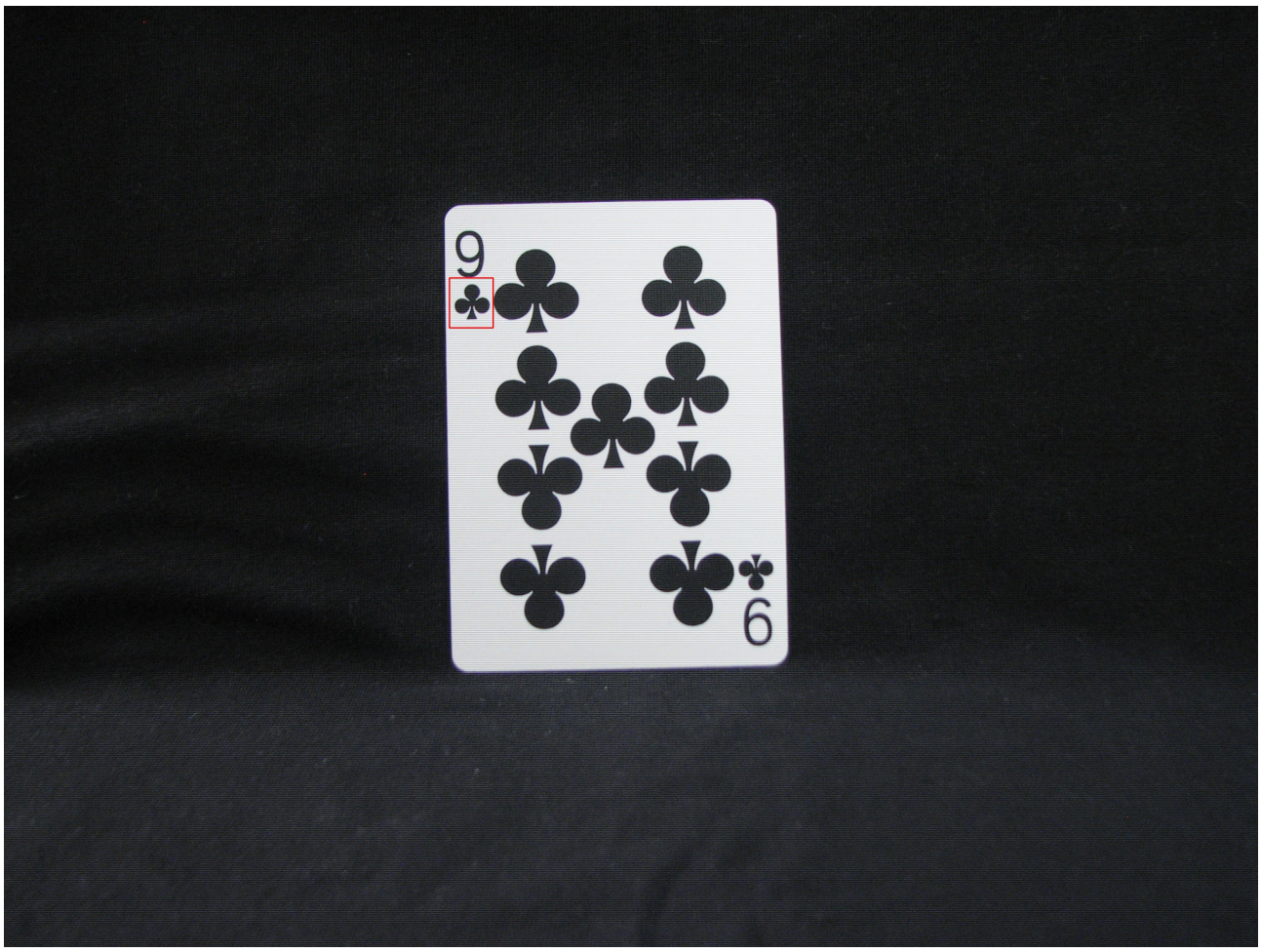


Slika 1: Ulazna slika za prvi primjer



Slika 2: Predložak za prvi primjer

Prvo učitamo ulaznu sliku i predložak, na osnovu njihovih veličina računamo varijable `rezcol` i `rezrow`. Te dvije varijable koristimo kako bismo napravili novu instancu klase `cv::Mat` u koju



Slika 3: Rezultat *template matching*-a u prvom primjeru

ćemo smještati rezultat *template matching*-a. To radimo preko instance funkcije `cv::Mat::create` koja alocira mjesto za novu sliku preko broja redova, kolona i tipa kojoj joj proslijedimo. Kada smo sve to uradili, možemo pozvati funkciju `cv::matchTemplate` koja redom prima: ulaznu sliku, šablon, sliku u koju smještamo rezultat *template matching*-a i parametar koji specificira metodu poređenja (postoji i peti parametar koji je predefiniran ukoliko ga mi ne proslijedimo, a to je upravo maska o kojoj smo ramnije pričali).

Nakon što smo dobili rezultat, koristimo funkciju `cv::minMaxLoc` koja nalazi najmanji i najveći element u nizu (u ovom slučaju instanca tipa `cv::Mat` i njihove lokacije. Trebamo ovdje naglasiti da ova funkcija ne radi sa multi-kanalnim nizovima (potrebno je iskorisiti funkciju `cv::Mat::reshape` onda). Kada smo našli minimalni i maksimalni element i njihovu lokaciju (u ovom kontekstu to znači gdje se najmanje i najviše šablon podudara sa ulaznom slikom), onda je potrebno da na lokaciji maksimalnog elementa nacrtamo pravougaonik koji odgovara šablonu na ulaznoj slici.

Koristimo funkciju `cv::rectangle` za to - ta funkcija prima niz na kojem trebamo nacrtati pravougaonik, lokaciju gdje trebamo nacrtati pravougaonik (lijeva gornja ivica pravougaonika), desna donja ivica pravougaonika, boju kojom želimo da nacrtamo pravougaonik (u primjeru je to crvena boja), debljina linije, tip linije i broj tzv. frakcionih bitova u koordinatama tačaka navedenih ivica. Rezultat je prikazan na slici 3 gdje vidimo da je uspješno nađen traženi šablon.

Prethodno pokazani primjer nam pokazuje da je korištenje *template matching*-a veoma neprecizna metoda za pronalaženje predmeta/lika na ulaznoj slici. Zapravo je pokazan najjednostavniji primjer - predložak je bio uzet sa ulazne slike, tako da je logično da će se naći

odgovarajuće podudaranje. Problem nastaje ukoliko imamo više različitih karata istog znaka koji su udaljeni različito od kamere ili ako su karte rotirane za neki ugao. U tom slučaju dolazimo do problema jer *template matching* samo provlači šablon iznad ulazne slike i gleda podudaranje - ali ako imamo zarotirani simbol, šta onda uraditi?

U nastavku će biti navedene funkcije koje su korisne upravo za prethodni slučaj:

- `cv::resize` funkcija kojom možemo smanjiti ili povećati predložak ukoliko je to potrebno.

```
void cv::resize(InputArray ulaz, OutputArray izlaz, Size dsize,
    double fx = 0, double fy = 0, int interpolacija =
    cv::INTER_LINEAR)
```

  Parametar `ulaz` predstavlja sliku čiju veličinu želimo promijeniti, `izlaz` je rezultantna slika nakon mijenjanja veličine, `dsize` je veličina izlazne slike, `fx` je faktor skaliranja po horizontalnoj osi, `fy` je faktor skaliranja po vertikalnoj osi, a `interpolacija` predstavlja način (metodu) interpolacije.

- `cv::getRotationMatrix2D` funkcija računa afinu matricu 2D rotacije (izvršena je neka linearna transformacija i/ili translacija).

```
cv::Mat cv::getRotationMatrix2D(Point2f centar, double ugao,
    double skala)
```

  Parametar `centar` predstavlja centar rotacije izvorne slike, `ugao` predstavlja ugao za koji rotiramo sliku (može biti pozitivan, u smjeru kretanja kazaljke na satu, ili negativan, suprotno od smjera kretanja kazaljke na satu) i `skala` predstavlja faktor izotropnog skaliranja (skaliranje invarijantno na bilo koji pravac).

- `cv::warpAffine` funkcija koja radi afinu transformaciju sliku.

```
void cv::warpAffine(InputArray ulaz, OutputArray izlaz,
    InputArray M, Size dsize, int flag = cv::INTER_LINEAR, int
    modus_granice = cv::BORDER_CONSTANT, const Scalar &
    vrijednost_granice = cv::Scalar())
```

  Parametar `ulaz` je ulazna slika koja se rotira, `izlaz` je rezultat nakon rotiranja, `M` je transformacijska matrica, `dsize` je veličina izlazne slike, `flag` kombinacija interpolacijskih metoda i opcionalnog *flag*-a `cv::WARP_INVERSE_MAP` koja znači da je `M` inverzna transformacija, `modus_granice` predstavlja metodu ekstrapolacije piksela i `vrijednost_granice` predstavlja vrijednost koja se koristi ukoliko je granica konstantna (postavljena je inače na 0).

Primijetit ćete da je implementacija ovih funkcija na realističnim slučajevima nepraktična. Postavlja se pitanje - za šta uopšte služi *template matching*? U realnosti se mogu koristiti za npr. testiranje kvaliteta u proizvodnji ili navigaciju mobilnih robota. U suštini, *template matching* je koristan za predloške koji nemaju tzv. jake osobine ili kada je većina predloška dio ulazne slike. Primijetit ćete prilikom rada na laboratorijskoj vježbi da je *template matching* spor, upravo zbog toga što se prelazi preko cijele ulazne slike sa predloškom - to mnogo usporava proces (šta ako imamo mnogo slika ili ako su slike velike?).

Napomene

Korištena tema za prikaz C++ koda na tutorijalima je `solarized`[1].

Literatura

[1] Ethan Schoonover. *Solarized*. `http://ethanschoonover.com/solarized`. Pristup stranici: 20. maj 2023. 2011.